

PART 5 – WAREHOUSING, ORDER PROCESS AND ADDITIONAL FEATURES

The final part of the primer extends the model that has evolved over the previous four sections and introduces some additional features of *FlexSim*.

The model's first extension is the addition of a small warehouse section that follows the Packing operation. That is, containers that have been finished and packed with components flow to racks in a warehouse where they are stored until they are used to fulfill demand. This introduces *FlexSim*'s Warehousing Module.

A second extension is the development of another model segment that adds a simple order-fulfillment process. The ordering process uses the containers that are finished and packed to meet arriving customer demand for packed containers.

In addition, a few of the remaining, but not all of the, basic objects (Multiprocessor, Transporter, Robot, and Crane) are introduced, as well as *FlexSim*'s Fluids Library.

The final section provides a textual summary of the model that has evolved over the course of the primer.

1. INTRODUCTION TO WAREHOUSE MODULE

FlexSim provides a robust set of tools for modeling warehouse operations. These tools can represent complex aspects of warehousing and storage systems. Since this is a primer, the warehousing capabilities are only introduced through a simple example. However, this should provide the basic concepts and a foundation for exploring the topic further. In any case, as has been stressed throughout the primer, it is always best to start simple, have a clear definition of the problem being addressed, and have clear operational objective(s) for developing the simulation model.

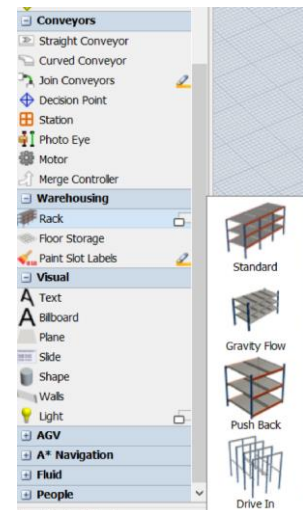
Objects are accessed through the Warehousing section of the Object Library. The most commonly used object, and the only one discussed in this primer is the Rack object. There are other types of racks in the Library for handling specific types of systems, e.g. Gravity Flow, Push Back, and Drive In Racks. While these differ from the Standard Rack, they are structured similarly.

The first time that a Warehouse object is dragged into a model, a Storage System object is added to the Toolbox. Most of the properties in the Storage System are for use in more advanced models; e.g., customizing schemes for addressing locations within a warehouse object, customizing visualization of objects, and triggers.



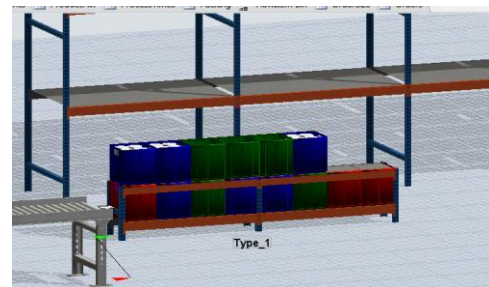
Begin with the basic model named Primer_4-3c, and **Save As** Primer_5-1. Basically make a copy of the model so that it can be customized and the original model is retained.

- Select the Standard Rack from the submenu in the Warehousing section of the object library, as shown in the figure to the right.
- Drag out a Rack object into the model.



In the figure to the right, the default Rack object is in the background and the customized Rack that will be used in this model is in the foreground. Obviously, among other things, the Rack will be resized for the primer model.

Also note in the figure, the Rack is storing all three types of containers.



The following steps customize the basic Rack to the Type_1 object used in this example.

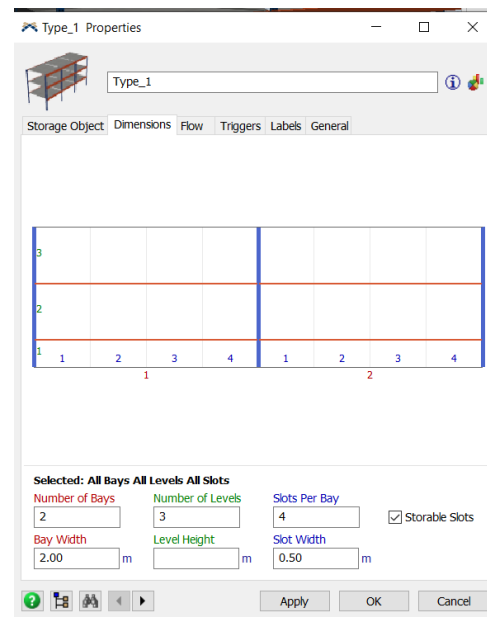
- Rename the general Rack object **Type_1**.
- On the General tab:
 - Set its color to red.
 - Change the size from the defaults of $x = 20.0$, $y = 1.5$, and $z = 8.40$ to $x = 4.0$, $y = 0.5$, and $z = 1.25$. Recall that a container is a 0.5 meter cube. Therefore, the capacity of the rack, in terms of the number of containers, is eight containers in the x direction, one in the y direction, and two in the z direction. This reflects what is shown in the figure above.

All of the default values for the properties on the Storage Object tab are not changed.

The settings for the Dimensions tab is shown in the figure to the right and explained below.

However, prior to customizing the tab, a few terms need to be defined. *Bays* are sections of a Rack that represent storage areas along the horizontal axis. Bays are subdivided horizontally into *Slots*. Bays also have *Levels*, storage areas in the vertical dimension. Each bay/slot/level combination is referred to as a *Cell*. Each cell may be considered either storable or not.

- As shown in the figure to the right, update the Rack properties for Type_1.
 - Set the **Number of Bays** to 2.
 - Set the **Number of Levels** to 3.
 - Set the **Slots Per Bay** to 4.
 - Set the **Bay Width** to 2.0.
 - Set the **Slot Width** to 0.5.



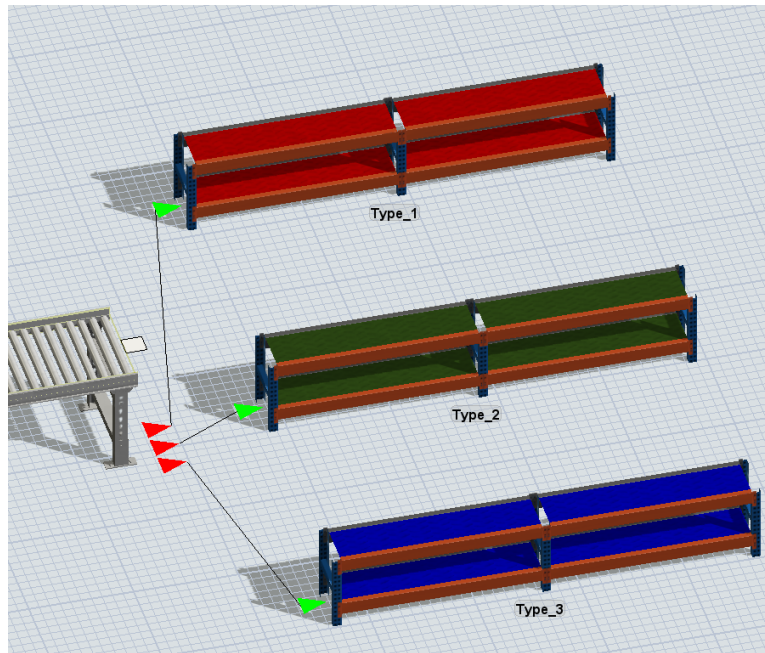
Now, what will likely result is what is shown in the diagram in the figure above, except that all of the **Level Height** values are the same, most likely 1.40. What we want is the bottom level to be 0.25 m. high and the top two levels to be 0.5 m. each. Each cell can be customized by selecting one or more cells in the diagram on the Dimensions tab to highlight them. While highlighted the **Slot Width** and/or **Level Height** properties can be changed.

- Select all of the slots on the bottom row/level and set **Level Height** to 0.25. Also uncheck the **Storage Slots** box since the lower level is not for storage; it is just to get the items off of the floor. Of course, no containers would be stored there anyway since a container's height exceeds the height of the cells in that level.
- Select all of the slots on the top two rows/levels and set **Level Height** to 0.5.

Thus, the 4 x 0.5 x 1.25 meter rack has 2 bays, 4 slots (in each bay), and 3 levels (2 of which can store items). Since a container is a 0.5 m. cube, the rack can store up to 16 containers. By default, containers are placed in the rack starting at Bay 1, Slot 1, Level 2. They are filled as follows: all slots in Bay 1 Level 2, then all slots in Bay 1 Level 3, then all slots in Bay 2 Level 2, and finally all slots in Bay 2 Level 3.

- Copy the Type_1 Rack object twice for the other two types of container.
 - Rename one **Type_2** and set its color to green.
 - Rename one **Type_3** and set its color to blue.
- Position the racks on the layout and connect the Conveyor's Exit Transfer to each Rack in the order of Type; i.e., Rack Type_1 should be connected to the Transfer's port 1, Type_2 to port 2, etc.
- Change the **Send to Port** property on the Exit Transfer to **By Expression** using the drop-down menu. The default value **item.Type** is what we want, to transfer from the Conveyor to a Rack based on the value of the container's label named Type.

The warehousing section of the model should resemble the figure below.



Since there is no Sink in the flow, if the model is run, then the racks will fill, containers will back up on the conveyors blocking the packing area and the finishing machines, and finally the containers' Queue at the beginning of the model will fill and all subsequent entering containers will be redirected elsewhere. The next section adds the logic to represent order fulfillment, where containers are removed from the Racks to meet demand.

2. ORDER PROCESSING SUBMODEL

Packed containers are pulled from the rack storage area to meet demand. Demand for containers of different types are based on incoming orders. The following is a brief summary of a simple representation of the ordering and fulfillment processes. This is followed by a description of how to model this in *FlexSim*.

2.1 Definition of the order-fulfillment process

Orders are released to production twice per 8-hour shift, halfway through (240 minutes) and at the end (480 minutes). This repeats for as long as the simulation runs.

Assume five orders are released at a time. The size of each order is a random variable, assuming there is a request for one container in an order occurs 10% of the time, two containers 15% of the time, three containers 50% of the time, four containers 15% of the time, and five containers 10% of the time. This is referred to as the order-size distribution. Of course, this distribution may change. Based on this distribution, an order consists of one to five containers and the average order size is 3.0 containers.

The mix of containers in an order is assumed to be the same as the previously assumed product mix: 20% are for Type 1 (red), 30% for Type 2 (green), and 50% for Type 3 (blue).

Once all of the containers are available for an order, the time to complete the order is normally distributed with a mean of 3 minutes and a standard deviation of 0.5 minutes.

For simplicity, no transport activities are modeled.

2.2 Implementation of the order-fulfillment process

The order size distribution is stored in a Global Table for easy access in the model and for editing and experimenting.

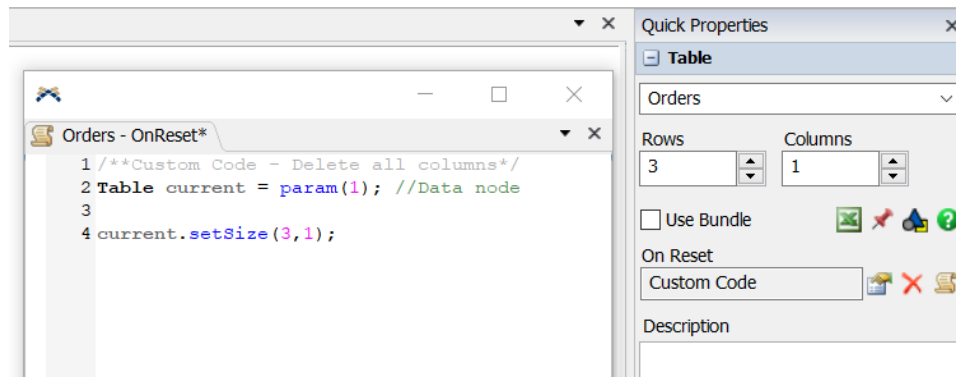
- Create a 5-row, 2-column Global Table named **OrderSize**, similar to the ProductMix table created earlier. In the model, order sizes will be randomly generated using the discrete-empirical distribution; again, in the same manner as the container's Type is generated at its Source. The first column contains percentages: **10, 15, 50, 15, and 10**, which correspond to the order sizes **1, 2, 3, 4, and 5**, which are stored in the second column of the table.

Another table is used to store the contents of each order. Each column corresponds to an order and each row contains the quantity of each type of container that is in that order. In other words, this file will contain a running history of all of the orders that occur during a simulation run. How this is used is discussed later, just create the base table now.

- Create a 3-row, 1-column Global Table named **Orders**. The default values of **0** in each cell are fine.

However, the Order table needs to be reset whenever the model is reset; i.e., all columns must be removed. In *FlexSim*, a table must have at least one column. Therefore, there is no option for this in the **On Reset** property in the table's Quick Properties window. Notice there is an option to **Delete All Rows, Clear All Cell Data**, etc., but no Delete All Columns option. Therefore, the logic needs to be specified using custom computer code.

- Select the scroll-like icon to the right of the textbox for the **On Reset** property. This will open a coding window as shown below.
- The first statement, **Table** current ..., provides a reference to the table. It does not need to be modified.
- Add the statement **current.setSize(3,1);** and be sure to enter the statement as shown, including the ending semicolon. Each coding statement must end with a semicolon. This statement is a *FlexSim* command that sets the size of the table to three rows by one column whenever the Reset button is pressed.



When the coding window is closed the **On Reset** property will show **CustomCode**. The code can be edited by clicking on the scroll-like icon to the right of the property.

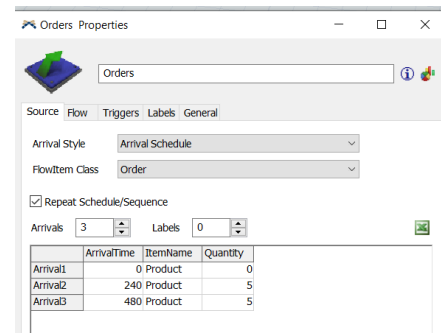
While not needed for this example, the interested reader can learn more about custom coding in *FlexSim* either via the *FlexSim User Manual* (accessed via the Help item on the Main Menu) or by another primer written by this author, entitled *A Primer of Coding in FlexSim 2017*.

Orders are represented as pallet-like flowitems and the containers are placed onto each order pallet as the order is fulfilled.

- Create a new flowitem for the order by duplicating the existing Pallet in the Flowitem Bin. All of the default values are fine, except:
 - Name the flowitem **Order**.
 - Add two number labels, **OrderNum** and **OrderSize** with default values of **0**.

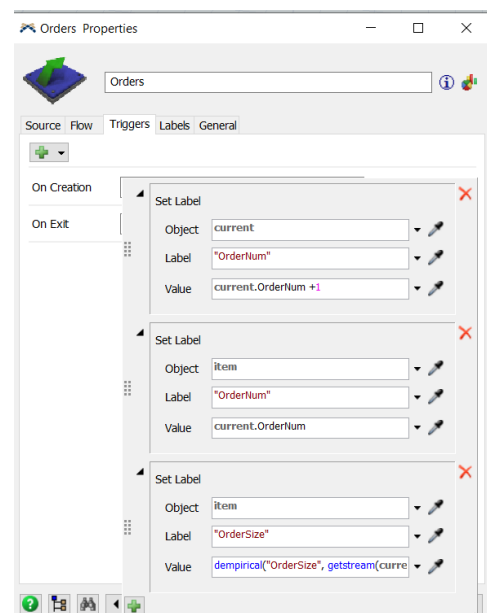
Orders are generated based on a repeating schedule, not randomly, from a Source object.

- Drag out a Source object from the Library and name it **Orders**.
- Change the properties on the Source tab as follows and as shown in the figure to the right.
 - Change the **Arrival Style** from the default **Inter-Arrival Time** to **Arrival Schedule**.
 - Check the box **Repeat Schedule/Sequence**.
 - Create an arrival schedule table with **Arrivals** (number of rows) set to **3**.
 - Change the **Arrival Time** and **Quantity** as shown in the table to the right. Five orders arrive at time 240 and 480. Since the table repeats, further orders arrive again at 720, 960, etc.



The **Arrivals Schedule** works as follows. The three rows are executed at the designated times and then Row 1 repeats 0 time (value of that row's Arrival Time) after the last row, then all rows are executed and Row 1 again executes 0 time after the last row, and the process repeats.

- On the Source's Labels tab:
 - Create a number label named **OrderNum** that has a value of **0**. This label's value will be the total number of orders created and is updated each time an order leaves the Source.
 - Check the box **Automatically Rest Labels**. Whenever, the model is Reset, the initial value of the label, **0**, is reset; i.e., zero orders are created at Reset.
- On the Source's Triggers tab, create an On Creation trigger, actually there will be three On Creation triggers that set labels, as shown in the figure to the right.
 - Select the **Data** category, then the **Set Label** option. As shown in the figure to the right and using the drop-down menu options, set the **Object** property to **current**, the **Label** property to **OrderNum**, and **Value** to **current.OrderNum + 1**.
This increments the total number of orders created.
 - Using the green + button, add the second **Set Label** trigger (selecting **Data** then **Set Label**). Using the drop-down menu options, select **item** for the **Object** property and **OrderNum** for **Label**. The **Value** is set to **current.OrderNum** from the **Labels** drop-down menu option.
This sets the created flowitem's order number; its value is the total number of orders created so far.
 - Using the green + button again, add the third **Set Label** trigger (selecting **Data** then **Set Label**). Using the drop-down menu options, select **item** for the **Object** property and **OrderSize** for **Label**. The **Value** is set using drop-down menu option **Statistical Distributions**, then **dempirical**. Change the parameter "MyTable" to "**OrderSize**".

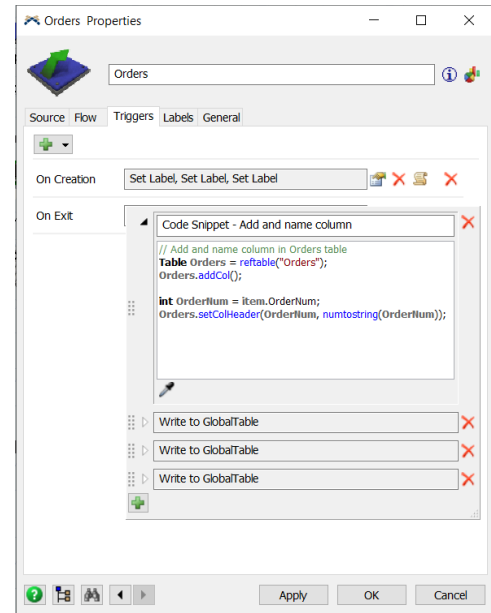


This sets the created flowitem's order size to a randomly-selected value that is based on the probability distribution defined in the Global Table OrderSize.

Again on the Source's Triggers tab, create an On Exit trigger, actually there will be four On Exit triggers.

- The first On Exit trigger, as shown in the figure to the right, is another small snippet of computer code. Every time a flowitem leaves the Source, it adds a column to the Global Table Orders with the header being the order number. Each line of code is briefly defined and explained.

- **// Add and name column in Orders table** is a comment statement that briefly describes what the snippet does. It is not a part of the logic, it is only included for clarity.
- **Table Orders = reftable("Orders");** sets a reference to the table that is being modified.
- **Orders.addCol();** is a *FlexSim* method that adds a column to the Orders table.
- The blank line is only for readability only.
- **int OrderNum = item.OrderNum** declares a local variable as an integer (**int**) and sets its value to the value of the flowitem's label that is being released from the Source. The label contains the total number of orders released so far.
- **Order.setColHeader(OrderNum, numtostring(OrderNum));** uses a *FlexSim* method to name the column just created to the item's order number. The header must be text so the order number must be converted from an integer to a string; this is done by the function **numtostring(...)**.

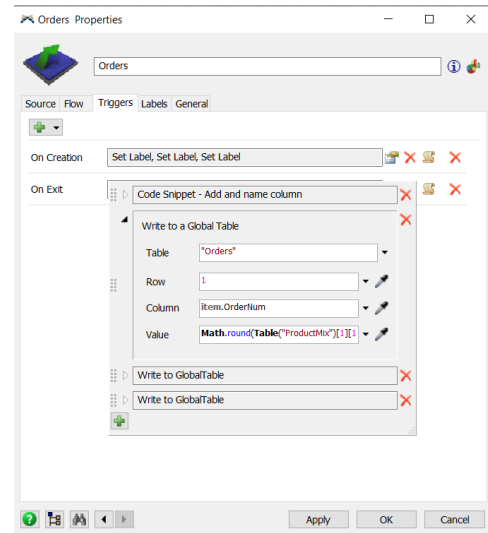


As mentioned above, a complete understanding of the coding is not needed; coding is a more advanced topic. However, the interested reader can learn more about custom coding in *FlexSim* either via the *FlexSim User Manual* (accessed via the Help item on the Main Menu or by another primer written by this author, entitled *A Primer of Coding in FlexSim 2017*).

The Source's On Exit trigger is also used to populate the Orders table. This is done by using three similar triggers, one for each item/container type.

- Using the green + button, add the second On Exit trigger by first selecting **Data** and then **Write to Global Table**. The interface is shown in the figure to the right and setting the property values are described below.

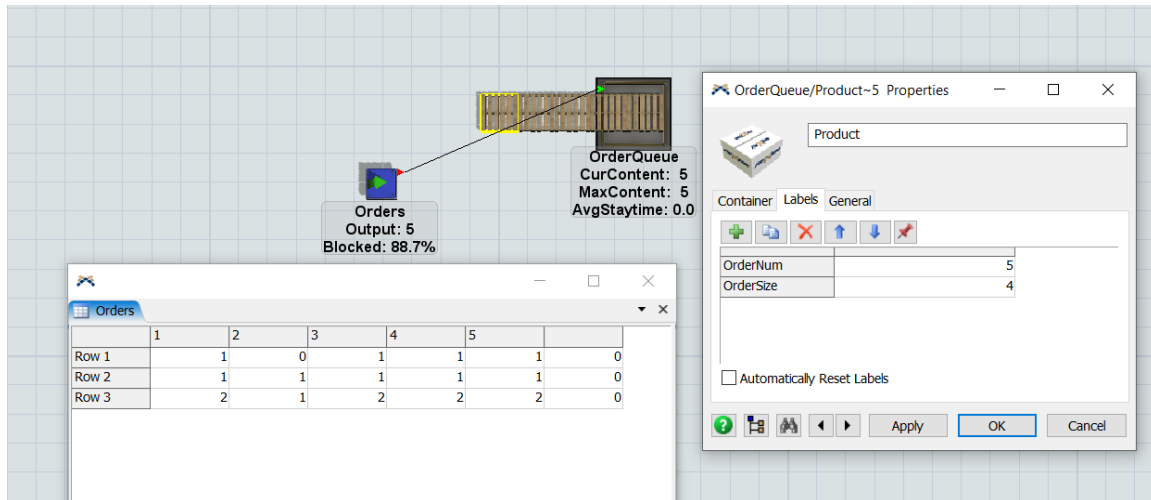
- For the **Table** property, select the table **Orders** from the drop-down menu.
- For the **Row** property, use the default value **1**. This is for item/container Type 1.
- For the **Column** property, select the table **item.OrderNum** from the **Labels** section of the drop-down menu. The column number in the table is the item's order number
- For the **Value** property, type in the following:



Math.round(Table("ProductMix")[1][1]*item.OrderSize/100.0)

This sets the number of item/containers of Type 1 that is in the order. It is the product of the percentage of Type 1s that is in the ProductMix table and the size of the order. For example, the percentage of Type 1s is 20% and if the order size sampled from the OrderSize distribution is 3, then the number of Type 1s in the order is $20 \times 3 / 100 = 0.6$. Since the number of items in an order must be an integer, the Math.round(...) method rounds the value to 1 in this case.

- Repeat the above steps for each other types of containers. In this case, create two more **On Exit, Write to Global Table** triggers. All of the values are the same except:
 - The **Row** value is **2** or **3**, depending on the item/container type.
 - The first **[1]** in the Math.round statement is changed to **[2]** or **[3]**, depending on the item/container type
- Drag out a Queue object and connect it to the Source.
 - Name it **OrderQueue**.
 - Set the **Item Placement** property, in the Visual section of the Queue tab, to **Horizontal Line**. This makes it easier to visualize the orders awaiting fulfilment
- **Reset** and **Run** the model. Since many changes were made, verify that the Source is working properly by checking the following. The model segment should resemble the figure below. In this case, the screenshot was made just after time 240 minutes, soon after the first set of orders was generated.
 - Check the labels on the orders/pallets – **OrderNum** should be sequential and **OrderSize** should vary between 1 and 5.
 - The Orders table should have a column for every order with header being the order number. The rows should be the number of containers of each type in that order.
 - Resetting the model changes the table back to a single column.

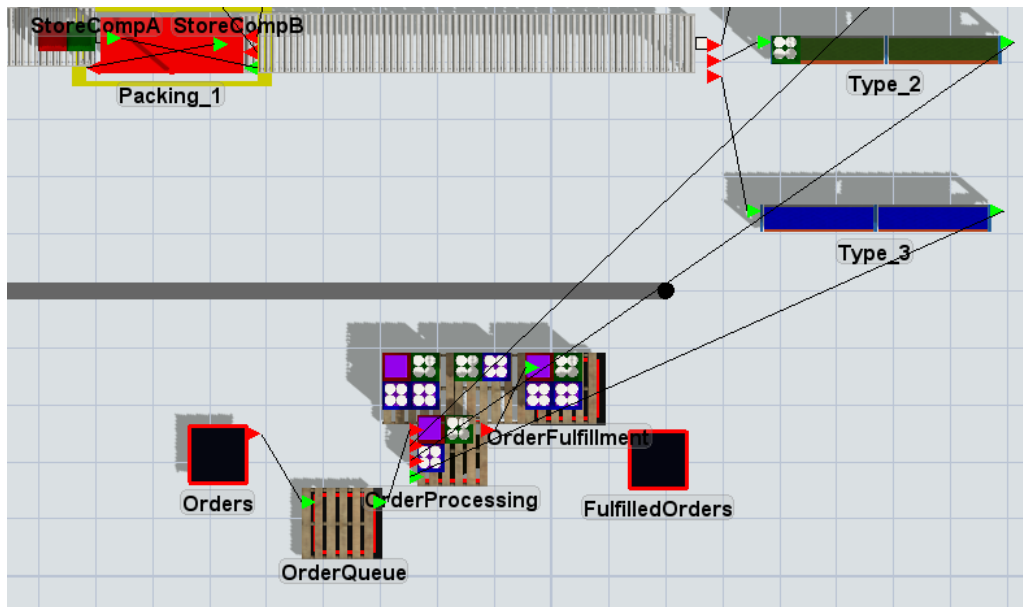


Note in the figure above the fifth order is selected in the 3D view. By double clicking on the selected pallet, the Labels tab shows that the value of the **OrderNum** label is **5**. The other label shows the size of the order to 4 items. Column 5 in the Orders table shows the breakdown of the four items in the order: one Type 1, one Type 2, and two Type 3s.

The Combiner object is used to fulfill the orders. Components from the Racks are combined onto an order pallet. The mix of components is determined from the Orders table.

- Drag out a Combiner object and connect it to the Queue (i.e., the submodel's flow is: Source → Queue → Combiner).
 - Name it **OrderProcessing**.
 - Set the **Process Time** property using the **Statistical Distribution** option, then select **Normal**. In the resulting interface set the **Mean** property to **3** and the **Standard Deviation** to **0.5**.
 - On the Triggers tab, use the + button to add an **On Entry** trigger. Then, select **Update Combiner Component List With Labels**. On the resulting interface:
 - Using the drop-down menu, for the **Table** property, select **Orders**.
 - Set the Label property to **"OrderNum"**.
- Connect each Rack to the Combiner in the order: Type_1 (red), then Type_2 (green), then Type_3 (blue).
- Drag out a Queue and connect the Combiner to it. This is only a temporary object to test the model - it will be replaced by a Sink. If the Sink is connected initially, we cannot visually verify that the orders are fulfilled correctly.
 - Name it **OrderFulfillment**.
 - Set the **Item Placement** property, in the Visual section of the Queue tab, to **Horizontal Line**. This makes it easier to visualize the orders that have been fulfilled.
- **Reset** and **Run** the model and verify that the order processing logic is working properly. Each item in the Order Fulfillment Queue should have the same mix of containers as in the Orders Global Table.

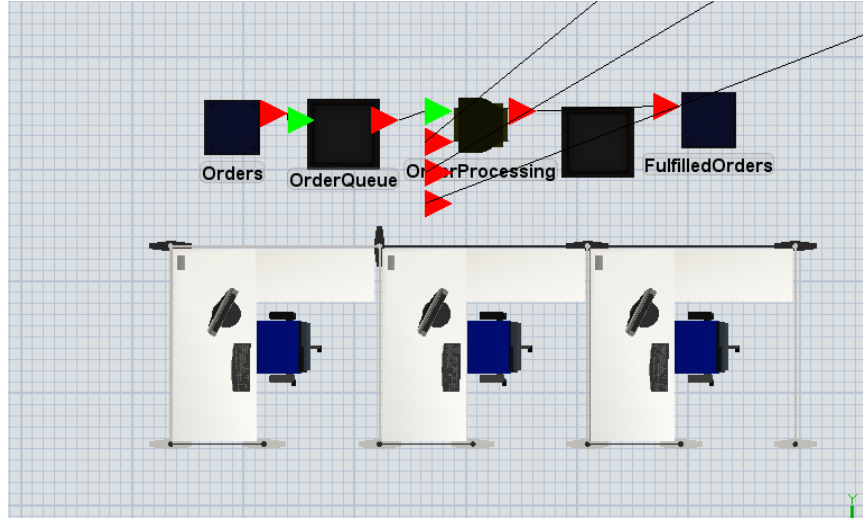
Note in the model shown below, the OrderFulfillment Queue is used to verify the model. At the time the screenshot was taken, three of the five orders that arrived at time 240 have been fulfilled (they are in the Queue), one is in process of being fulfilled and one is awaiting processing.



	1	2	3	4	5	
Row 1	1	0	1	1	1	0
Row 2	1	1	1	1	1	0
Row 3	2	1	2	2	2	0

Compare the Queue contents with the first three columns in the Orders table. For example, the first completed order in the OrderFulfillment Queue contains one red container, one green, and two blue; this corresponds to the first column in the table. Orders two and three in the Queue also match the orders in the table.

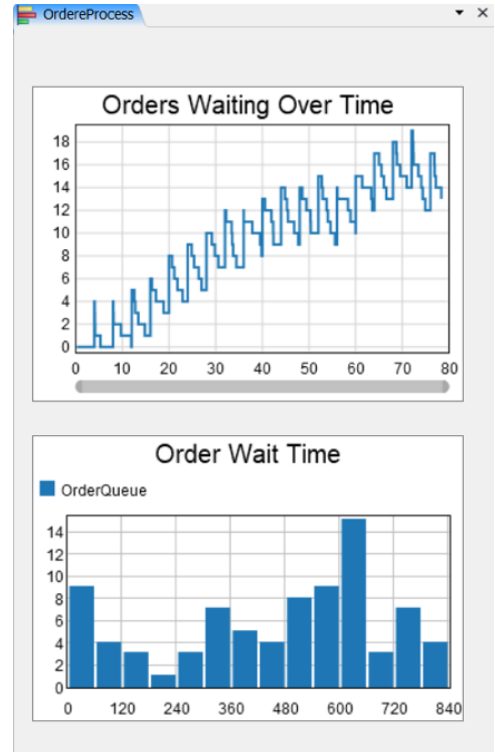
- Once the model is verified, drag out a Sink object and name it **FulfilledOrders**. Replace the connection from the Combiner to the OrderFulfillment Queue with a connection to the Sink.
- The order process can be placed anywhere in the model and the objects can be made small, or hidden, since the way order fulfillment is modeled here it is not a physical process. For example, the Source, Queue, Combiner, and Sink can all be set to 0.5 meter square by 0 meters high, as shown in the figure below.



The order process objects can be hidden by covering them with a visual object.

- Drag out an instance of the **Shape** object (gray cylinder) from the Visual section of the Object Library.
 - Rename the object **OrdersOffice**.
 - On the General tab. Select the drop-down menu for the **3D Shapes** property (fs3d\General\Cylinder.3ds) and **Browse**. Locate the Sketchup Files folder (should be in your FlexSim Models folder) and select the **Office.skp** file.
 - As shown in the figure above, resize the object so that it is 6m in the x direction, 2m in the y direction, and 2m in the z direction.
 - Hide the order process logic by covering it with the office shape.

- Create a Dashboard named **Order Process**, as shown in the figure to the right and as defined below.
- On the Dashboard, add a Content plot from the Content templates section of the Dashboard Library and name it **Orders Waiting Over Time**.
 - On the Options tab, use the green + button in the Objects section to select the object **OrderQueue**.
 - On the Settings tab, using the drop-down menu, change the **Time Axis Mode** to **Show Duration** and change the units from the default **Seconds** to **Hours**.
- On the Dashboard, add a Staytime histogram from the Staytime templates section of the Dashboard Library and name it **Order Wait Time**.
 - On the Options tab, use the green + button in the Objects section to select the object **OrderQueue**.
 - On the Settings tab, change **Bar Mode** to **By Bucket Width** and set the **Bucket Width** value to **60** so that each bar represents one hour of wait time.
 - On the Text tab, change the **Precision** property from **2.00** to **0**



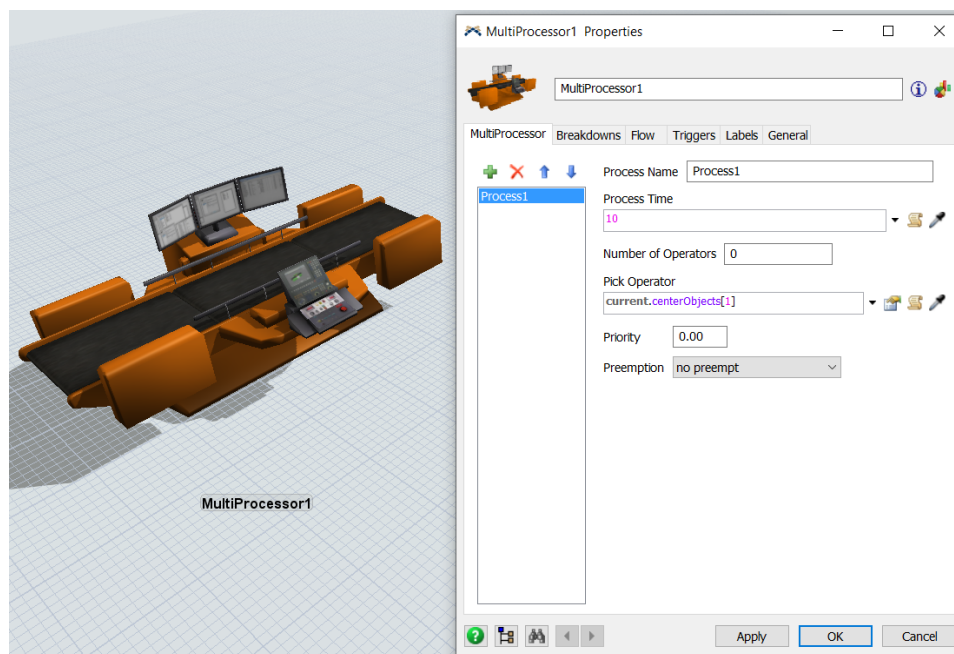
Note that based on the Dashboard plots, the system seems to be out of balance. This is evident in the time-series plot where the number of orders waiting is generally increasing over time. The histogram indicates that many of the order wait times are quite large - many are over ten hours (600 minutes)!

3 OTHER FLEXSIM OBJECTS AND CAPABILITIES

This final section introduces some additional modeling constructs that are used to model other system features than those described in the primer. These are certainly not all of the additional capabilities in *FlexSim*; they are a sampling that are intended to identify and introduce some other features. The additional constructs include other objects: a Fixed Resource object, the Multiprocessor, and three more Task Executer objects, the Transporter, Crane, and Robot. This section also includes another library of objects, the Fluids Library.

3.1 Multiprocessor object

The Multiprocessor is a Fixed Resource similar to the Processor, Combiner, Queue, etc. Its 3D shape and Multiprocessor properties tab is shown in the figure below.



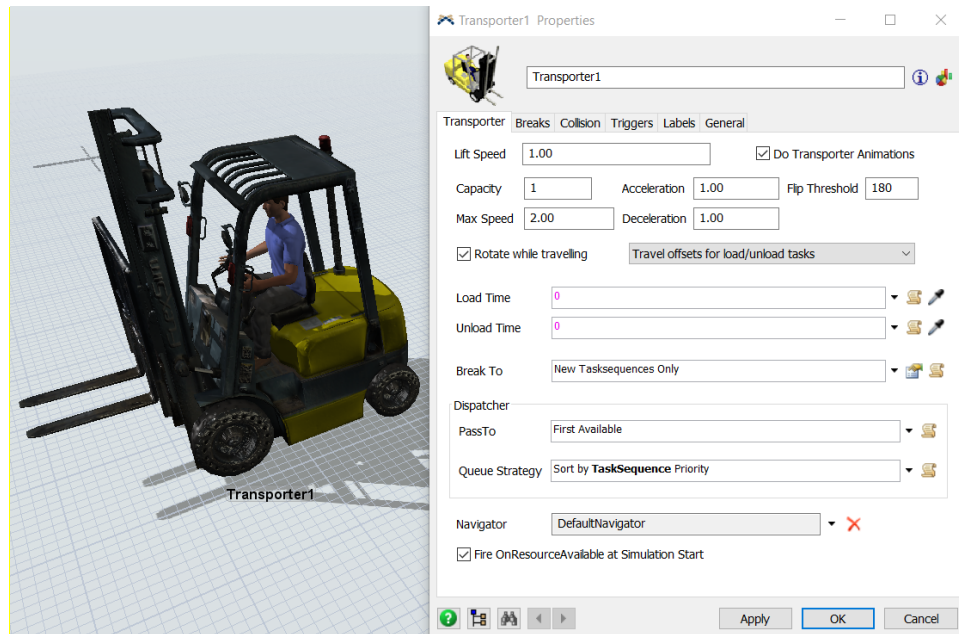
The object puts a single flowitem through a sequence of separate processes; thus, only a single flowitem is in a Multiprocessor at a time. Each process has its own Process Time, and means to set the process time, as well as use one or more Operators.

Different types of flowitems can be processed by different processes. For example, if a Multiprocessor has four process steps defined, say A through D, then flowitem Type 1 might use process steps A and D and flowitem Type 2 might use process steps B and C. In this case, the process time for process B and C would be 0 for Type 1; and, the process time for process A and D would be 0 for Type 2.

It is important to note that if when a flowitem completes a process step and then another flowitem can enter that step, then the Multiprocessor is not the object to use to model this situation. In this case, simply use several Processor objects connect in sequence.

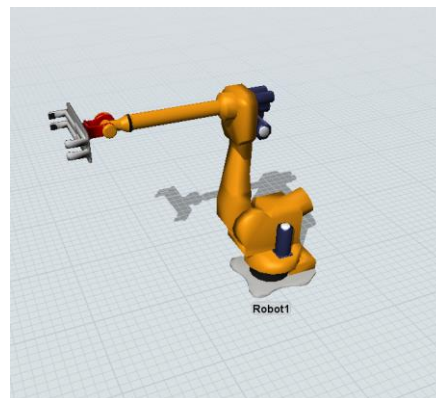
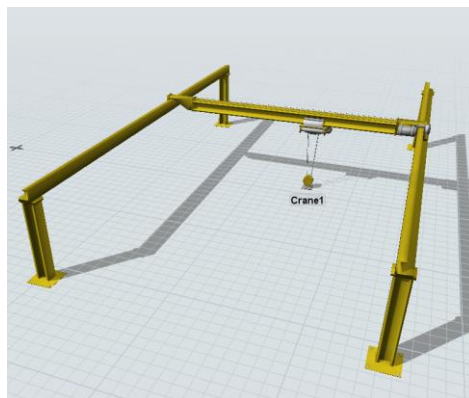
3.2 Transporter object

The Transporter object is a task executor, similar to the Operator object used in the primer. The basic 3D object, which resembles a fork truck, is shown in the figure below, along with the main tab on the object interface. Note the similarities between the Transporter properties and the Operator object properties. One difference between the objects is that the forks on the Transporter raise and lower to load and unload flowitems.



3.3 Robot and Crane objects

The Robot and Crane objects are both task executors, similar to the Operator object used in the primer and the Transporter introduced above. These basic 3D objects are shown in the figure below.



As mentioned previously, all task executors have the same functionality, primarily to interact with Fixed Resources and move flowitems.

The Robot object is a special transport that lifts flowitems from a starting point and places them at an end point. Generally the Robot's base does not move, but it has six rotating joints that transport flowitems.

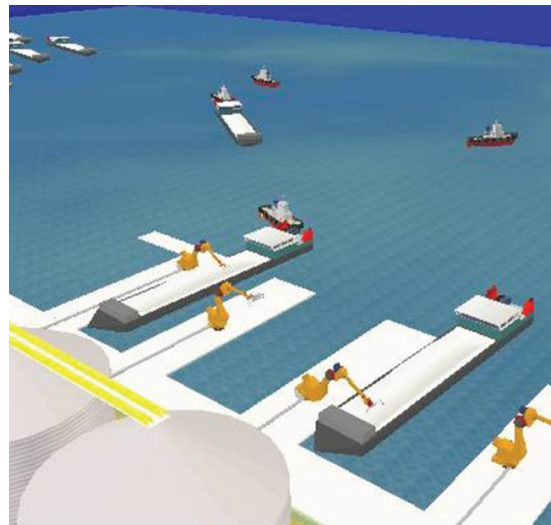
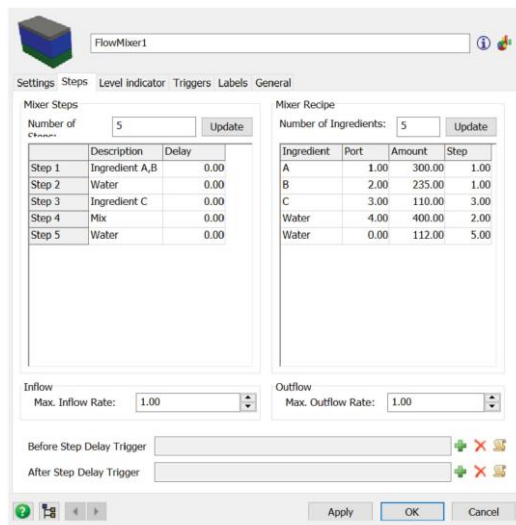
The Crane object is also a special transport that simulates the operation of rail-guided cranes, such as gantry, overhead, or jib cranes. Its functionality is similar to a Transporter, but with modified graphics. By default, the crane picker rises to the height of the crane object after picking up or dropping off a flowitem, then travels to the next location. Of course all of these actions can be customized.

3.4 Fluids Library

Fluid objects are found in the Fluids section of the Drag-and-Drop Library. These objects provide a means to model systems where at least some of the states of the system change continuously over time rather than at discrete points in time. These objects enable *FlexSim* to simulate continuous systems representing fluid flows (e.g., water, oil, and gases), particulate flows (e.g. powders, cereal, and potato chips), and high-speed operations (e.g. bottling). A fluid (or continuous) flow is usually described in terms of units per time, such as gallons per minute.

Example Fluid objects include: Fluid Generators, Fluid Tanks, Fluid Terminator, Fluid Mixer, and Fluid Conveyors. Many of the fluid object are analogous to Discrete objects that have been discussed at length in this primer, such as Sources, Queues, Sinks, Combiners, and Conveyors. There are also objects that convert fluid entities into discrete entities and visa versa. Models that include a mix of both discrete and continuous elements are referred to as hybrid models.

The following figure shows an example interface for the Fluid Mixer object on the left and a full model using Fluid objects on the right. The Mixer object uses both a Step and Recipe table to define batch-processing operations, which are common in most fluid systems. The full model is used to assess operations and resource requirements (e.g. docks and tugs) at a grain-loading facility where barges are loaded with grain from silos.



Congratulations! You have successfully completed the *FlexSim Simulation Software Primer*. Hopefully you now have a better understanding of how to build *FlexSim* simulation models to solve problems and make better decisions through the power of discrete-event simulation and the robust capabilities of *FlexSim*.

4 SUMMARY OF THE PRIMER MODEL

This final section provides a textual summary of the model that evolved throughout the primer. The final model is the result of a development process that progressed from a very simplified version of the system to one that more closely represents the real operations.

Recall that Dobre Products Limited (DPL) is planning to reuse an area in one of its production facilities to finish and pack containers for distribution. The new production area will finish various types of containers and then the containers are packed, where the contents depend on the type of container. The packed containers are then moved to a warehouse area where they are used to fulfill customer demand.

In this initial model, which would likely continue to evolve and expand as the design process continues, it is assumed that there will only be three types of containers packed with two types of components. Of course, based on the manner in which the model was developed, it is quite easy to expand the model to consider a larger number of products and components as well as increased production resources, people and equipment.

The basic units of measure are meters and minutes.

The key properties of the three products are summarized in the following table.

Property	Product/Container					
	1		2		3	
Size (l x w x h)	0.5	x 0.5x 0.5	0.5	x 0.5x 0.5	0.5	x 0.5x 0.5
Percent of product mix	20		30		50	
Color	Red		Green		Blue	
Finishing Time, average	15		20		30	
Packing Time, average	2.5		3.33		3.83	
Quantity of Components A, B packed	2, 0		0, 4		1, 4	

The key properties of the two components that are packed into the containers are summarized in the following table.

Property	Component	
	A	B
Size (l x w x h)	0.4 x 0.4 x 0.2	0.2 x 0.2 x 0.2
Shape	Box	Cylinder
Color	Purple	White
Batch Size	24	60
Initial Inventory	12	30
Reorder Point	6	20
Time to Receive and Order	120	120
Time to unload, per batch	1	1
Time to unload, per item	0.05	0.05

The containers arrive to the finishing area (upstream boundary of the model) with an average time between arrivals of 15 minutes. (Alternatively, the average arrival rate of four containers is per hour.) The times between arrivals are triangularly distributed with a minimum time of 5 minutes, maximum time of 30 minutes, and most-likely time of 10 minutes.

- There are no breaks in the arrival pattern; i.e., there is no downtime in the upstream operation.
- The type of container that arrives is random and is based on the percentage of product mix; i.e., there is no batching of containers in upstream operation.

Arriving containers wait to be placed on a finish machine in an area that has the capacity to store five containers; the size of the storage area is 1.1 meters by 8 meters. If when a container arrives and there is no available space, it is diverted to another area in the facility and is no longer considered in the model other than it is counted as a “diverted item.”

Containers are loaded onto one of two finish machines by a single finish operator. The operator uses the Shortest Processing Time (SPT) rule where the container that is the quickest to process on a finish machine is loaded first. However, Dobre does not want items to wait in the buffer too long; therefore, they use the SPT rule unless a container has waited more than 30 minutes or some other specified threshold value.

Currently, there are two identical finish machines that can process any type of container. Each machine is 3 meters square and 2.5 meters high. The process time depends on the type of container. A setup is needed before processing if the container being loaded is different than the one that just finished processing. The setup is performed by the finish operator and the time to perform the setup is 1 minute. Once loaded, the machine processes the container without the operator. After processing, containers move onto a conveyor that transports it to the packing area.

Finish machines are subject to two types of downtime. The first type of downtime is for a quality check where each machine pauses for 15 seconds every ten minutes to upload data. The second type of downtime is when there is some type of failure within the machine. The time between failures is a random variable that is exponentially distributed with a mean of two hours. The time to repair the problem is also a random variable that is uniformly distributed between 5 and 15 minutes. For the failure downtime, the finish operator performs the repair. Failures can only occur when a machine is running.

After the finish operator loads a container into the finish machine and performs a setup operation, if needed, the operator travels to a kiosk and records some production information. The recording time is uniformly distributed between 30 and 90 seconds. Once the information is logged in, the operator is then free to perform another task.

The finish operator travels at an average speed of 60 meters per minutes and takes 30 seconds to load or unload any item. The operator takes two 15-minutes breaks per shift, one after two hours and one after six hours into the shift. The operator also takes a 30-minute meal break in the middle of the eight-hour shift. For all breaks, the operator travels to a break area outside of the finish and packing areas.

Once a container arrives at the single packing station, it is loaded with the number and type of components that are specified in the table above and then moved by conveyor to the warehouse. Once in the warehouse area, each container is placed on a rack; there is one rack for each type of container. The packing station follows the same break schedule as the finish operator. The speed of all conveyors is 90 meters per minute.

Components to be packed into the containers are delivered according to a Reorder-Point type of inventory system. That is, when the on-hand quantity of a component in the packing area drops to a specified reorder point, a batch of that component is ordered. When a batch of components arrives, the finish operator unloads the batch. The time to unload a batch is one minute plus three seconds per component.

Customer orders for containers arrive throughout the day, but are released to production twice per shift, once midway through the shift and once at the end. Five orders are released at each of those times. The number of containers per order averages three, but varies between one and five (10% of the orders are either one or five containers, 15% are two or four, and 50% of the orders require three containers). The type of container varies, but is the same as the product mix defined in the first table above.

Orders are fulfilled in the warehouse in the order in which they are released. Once all of the containers for an order are gathered, the time to complete an order is normally distributed with a mean of three minutes and a standard deviation of one-half minute.

The following are the key performance measures of interest to Dobre. Adjusting production parameters, such as reorder point and quantity, process times, equipment speeds, etc. should help them better design the facility.

- The time it takes to fulfill an order in production, from release to production to release to shipping.
- The number of orders waiting to be fulfilled in production.
- The number of components in inventory.
- Number of containers awaiting finishing and packing.
- Utilization of finish machines and finish operator.
- ...

Each simulation run is for 80 hours and when running experiments, each scenario is replicated 20 times.

EPILOGUE

This primer focused on the modeling and analysis of operations systems in order to understand and analyze their dynamic behavior and performance (operations dynamics). The reader is led through detailed, step-by-step instructions for building a comprehensive simulation model. A screenshot of the final model is on the next page. The model evolves and builds upon simpler models using a sequential model-development process. In addition to describing the mechanics of building a model, the primer demonstrates the power of using discrete-event simulation in general, and *FlexSim* in particular, to support the decision-making and problem-solving processes. As the model evolves insight and rationale are provided and good modeling and analysis practices are introduced – this is to demonstrate that model building is not just carrying out rote commands.

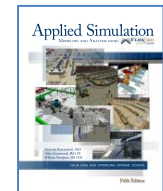
While all of *FlexSim*'s capabilities are not presented, the primer raises awareness of many of the more advanced features available in *FlexSim* without covering the details at that time. Successfully completing the primer should provide the reader with a solid foundation to explore topics in more detail in the *FlexSim User Manual*, tutorials, videos, blogs, *FlexSim Answers*, etc.

It was also beyond the scope of the primer to discuss, or even introduce, all of the topics and methods that are associated with simulation modeling and analysis. Therefore, it is suggested that the interested reader consult the following or other more general references on simulation.

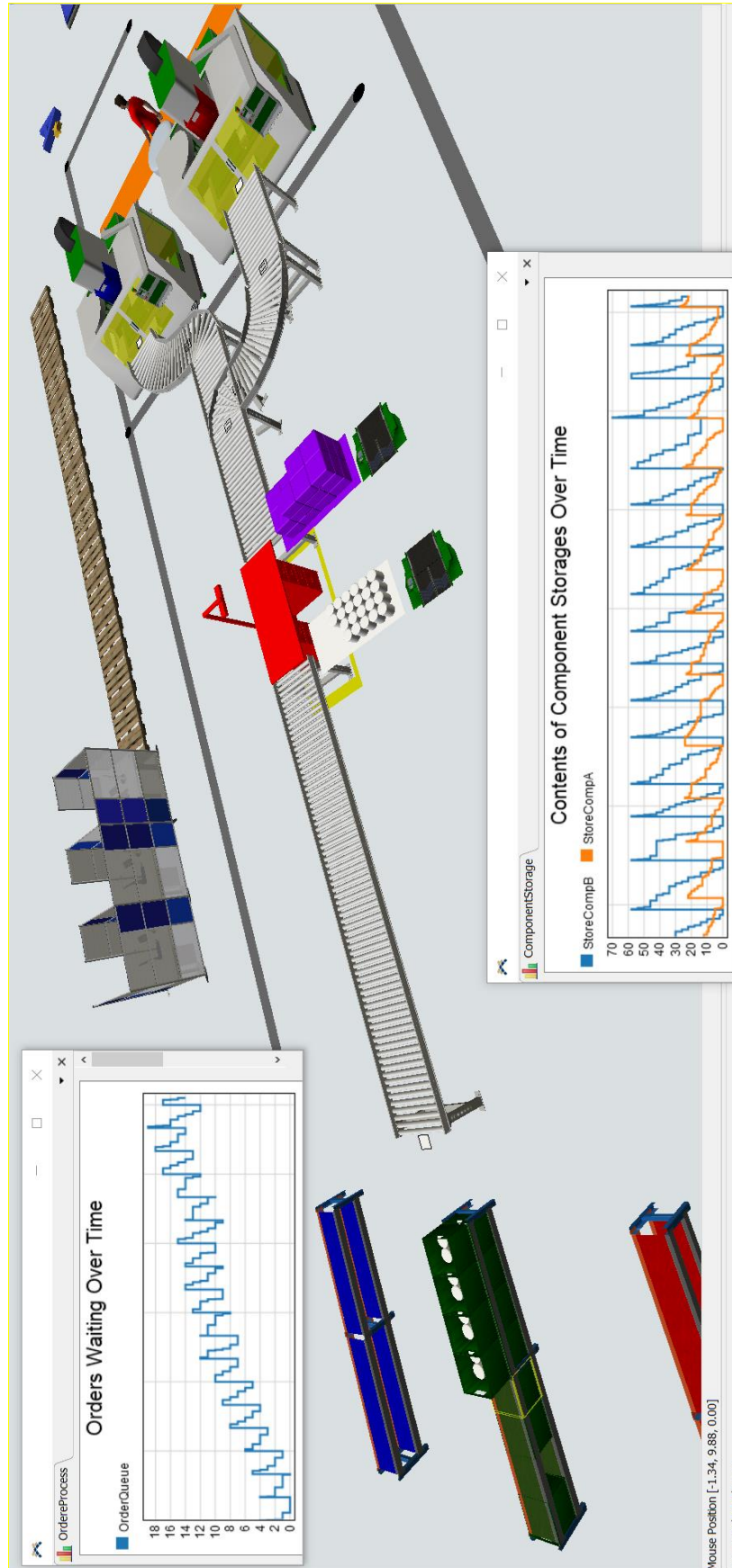
Greenwood, A. *Simulation Primer*, FlexSim Software Products, Inc., 2019.



Beaverstock, M., Greenwood, A., and Nordgren, W. *Applied Simulation Modeling and Analysis Using FlexSim*, 5th Edition, FlexSim Software Products, Inc., 2017.



Since improvements cannot occur in a vacuum, the author welcomes comments and suggestions; please send them via email to allen.greenwood@flexim.com.



APPENDIX - LIST OF MODELS

This appendix provides a list and brief description of the 23 models that are discussed in the primer. Most of them are used as part of a sequential model-development process that evolves from a very simple representation of the system being considered to a more complex, more realistic representation. A few of the models are smaller “study” models that are used to understand, test and/or validate a method or concept in isolation before incorporating it into the main model.

As has been mentioned several times, all of the models listed below, along with supporting files (a layout file and two 3D representations of work areas in the example), are available in a resources folder that can be obtained at <https://flexsim.com/primer-files>.

<u>Filename</u>	<u>Brief Description</u>
MyFirstModel	Part 1, Section 4. Basic objects and connections. Simple, single-server queueing system using all default values.
Primer_1-1	Start of the primer models. Same as MyFirstModel..

Note: All subsequent primer models begin with the previous model and add further customization to the model in order to more closely represent the system described in Part 1, Section 2.

Primer_1-2	Part 1, Sections 5 (Basic Fixed Resource Objects and Customization) and 6 (Basic Model Output). • Further description and customization of the following basic objects: ○ Source - inter-arrival times to triangular distribution; flowitem class to Tote; On Creation trigger to set Type label and color. ○ Queue – maximum contents and item placement properties. ○ Processor – process time based on product type; set-up time based on whether product changes. ○ Sink ○ Flowitem Bin • Run Time and Run Speed. • Object statistics – on-object stats, Quick Properties stats. • Dashboard – time-series plots of contents and staytime histogram.
Primer_2-1	Part 2, Sections 1 (Changing 3D Graphics) and 2 (Alternative Routing) • Importing graphics – layout and object shapes; changing size, location, rotation, and shape of objects; reference points. • Alternative routing rules, Flow tab.

Primer_2-2	Part 2, Section 3 (Task Executors) • Dispatcher to control Task Executor (Operator) • Operator transports flowitems; Operator performs setup operation on Processor. • Operator properties: speed, load/unload times, location on model reset. • Creation and use of Path Networks to control Operator travel.
Primer_2-3	Part 2, Section 4 (Conveyors) • Straight, Curved, and Join conveyor objects; Transfers. • Conveyor types and properties.
Primer_2-4	Part 2, Section 5 (Global Tables) • Global table to store and use process times. • Global table to store and use product types (percent product mix).
Primer_2-5	Part 2, Section 6 (Downtime) • Time Table to control time and duration of Operator breaks; change color and travel to break location. • Reliability, using MTBF/MTTR tool to represent downtime for a fixed-interval quality check activity that is clock-dependent and requires no repair resource. • Reliability, using MTBF/MTTR tool to represent downtime for a randomly occurring machine breakdown that is state-dependent and requires an Operator for repair. • Chart the impact of downtime using a pie chart of utilizations (Processors and Operators) on a Dashboard.
Primer_3-1	Part 3, Section 1 (Combining and Separating Flowitems) • Create and customize flowitems in the Flowitem Bin. • Use scheduled arrival of batches at a Source, rather than inter-arrival times.
Primer_3-2	Part 3, Section 1 (Combining and Separating Flowitems) • Fixed-recipe Combiner (default) to simulate packing operation. • Global Table to define a Combiner's combining recipe; i.e., by flowitem type. • Extend a Global Table to contain packing times by product type in addition to process times.

Primer_3-3	Part 3, Section 1 (Combining and Separating Flowitems) • Time Table to set breaks for the packing station. • Global Table to set the inter-arrival time at a Source. • Object Label to identify the type of component it generates. The item's Type label is set to the same value as the object's label. • Separator to convert the arrival of a batch to the number of flowitems in the batch. Batch size is stored in a Global Table, along with other parameters. • Operator to unpack a batch of items. • Process Time, the time to unpack a reorder of components is based on the type of product and the batch size. The time is based on an expression of Global Table values. Introduces coding in <i>FlexSim</i>. • Extended path network, including curved paths. • Display the contents of a buffer over time in a Dashboard.
A-StarStudy	Part 3, Section 2 (Controlling Task Executer Travel Paths Using A*) • A* is an alternative means to path networks for controlling task executer travel. This simple model illustrates the basics of how A* works and how to incorporate it into a model. • Illustrates the use of a study model to test a concept and understand a model's behavior in an isolated environment.
Primer_3-4	Part 3, Section 2 (Controlling Task Executer Travel Paths Using A*) • Set up the model to use the A* algorithm in order to control Task Executer (TE) travel. A* is an alternative means to using Path Networks for TE travel. • Specify which model objects the TE needs to avoid; add Barrier and Divider objects. • Show travel paths via a Heat Map. • Define and access Model Views. • Re-order the contents of a queue when an item arrives based on an item label
Primer_3-5	Part 3, Section 3 (Using an Item List for Complex Routing Logic) • Simple re-ordering rules for Queues. Sort Queue based on type of flowitem. • Basic introduction to, and definition of, the Lists modeling tool, including defining Fields, using Push To and Pull From Lists constructs, and viewing a List's contents and backorders. • In the example, a List is used to re-order a Queue based on two criteria – time in queue and item type.

- Primer_3-6a
- Part 3, Section 4 (Experimentation in *FlexSim*)
- Basic setup of the Experimenter – defining decision/independent variables, performance measures, and run time parameters; generating and analyzing standard output.
 - Assess the affect of buffer size on performance (number of redirected arrivals to insufficient storage).
- Primer_3-6b
- Part 3, Section 4 (Experimentation in *FlexSim*)
- An extension example to assess the affect of an additional variable, product mix (percentage of each product type), on another performance measure (system throughput).
- Primer_4-1
- Part 4, Section 2 (Introduction to Process Flow – Creating Initial Inventory)
- Basic Process Flow environment.
 - Simple Process Flow Model to create flowitems in all Queues (initial inventory) using Scheduled Source and Create Object activities. Parameters set using Global Table lookups.
 - Use an Array-typed Global Variable to generalize the above process so that inventory can be initialized in any number of Queues.
- Primer_4-2
- Part 4, Section 3 (Modeling Inventory Reorder Policy Using Process Flow)
- Create a label on each Queue denoting what component it stores.
 - Create another Global Variable that contains a pointer to the Separator object for each component.
 - Create a Group of Queues that store components. Use the Group tool to help develop a general process for managing inventory in any number of Queues.
 - Check for the need to reorder by “listening” for exits from Queues using the Event-Triggered Source activity. Using the Assign Labels activity to obtain and save information on the current state of the system. Compare the number in Queue with the re-order point using the Decide activity.
 - If the decision from the process described above, requires a reorder, use: the Delay activity to implement the reorder time, Change Visual to change the object’s color denoting a reorder has been placed, and the Create Object activity to place the ordered item in the model.
- Primer_4-3a
- Part 4, Section 4 (Custom Task Sequences in Process Flow)
- Addition of a 3D model object that represents a kiosk and including it on the Operator’s travel path. It is used later as part of an example that creates a custom task sequence in Process Flow to control an Operator’s behavior.
 - Increase incoming production rate by 25% by adjusting the Source’s inter-arrival time distribution parameters.
 - Miscellaneous model housekeeping actions: confirm parameter settings, revert the model back to using the path network for Operator travel that had been replaced by the A* algorithm, add curvature to some of the network paths to avoid objects, and hide the imported layout.

Simple TE Process Flow	Part 4, Section 4 (Custom Task Sequences in Process Flow) • Create a simple custom task sequence in Process Flow and compare its behavior to the default travel task sequence that is used by Operators. • Illustrate the relationship between events in the 3D model and Process Flow.
Primer_4-3b	Part 4, Section 4 (Custom Task Sequences in Process Flow) • Apply the concepts learned in the study model to the main primer model. • Add a custom task sequence to the finish operator that involves the operator entering information at a kiosk after loading a finish machine.
Primer_4-3c	Part 4, Section 4 (Custom Task Sequences in Process Flow) • Add the Processor's setup operation to the custom task sequence created in Process Flow in the previous model. The setup task is added to the transport and kiosk work task sequence. Requires adding a label to the Processor that is the flowitem's type that just finished processing.
Primer_5-1	Part 5, Section 1 (Warehouse Module) and 2 (Order Processing) • Use customized Rack objects to store components. • Introduce a simple order-generating and order-processing model. Orders have a randomly-generated mix of components based on an empirical distribution. Orders are fulfilled from the Racks. Complexity arises in writing each order's mix of components to a Global Table that is used by a Combiner to fulfill the order. Small custom code snippets are required. A new dashboard of performance measures is added. The order processing logic is "hidden" in the model using a Visual object with an imported office-like shape.

APPENDIX - GLOSSARY

This appendix provides definitions for some of the key terms used in the primer.

3D shape	A visual representation of an <i>object</i> that is defined through an imported graphics file. The size, location, color, etc. is defined through the object's General tab user interface.
A* Algorithm	A heuristic search method used to find the shortest path between objects considering barriers or obstacles and no-travel zones.
A* Module	A <i>FlexSim</i> tool for creating barriers, and identifying which fixed objects are barriers, in a model. The barriers influence and restrict Task Executer (TE) travel paths between objects. In contrast to <i>Path Networks</i> , which also controls Task Executer movement, A* tells the TE where <u>not</u> to travel, compared to path networks, which restricts where a TE can travel.
Activity (Process Flow)	Logical operations or steps in a logical process that are the building blocks of <i>FlexSim</i> 's Process Flow. They are analogous to objects in 3D in that they (1) are dragged from their library onto a Process Flow modeling surface or workspace and (2) have properties that define their behavior.
Activity Set (Process Flow)	Preconfigured activities bundled together that model a basic People Module task, such as Walk then Process, Escort then Process, Wait then Process, etc.
Address (Rack)	Identifier of where an item is located/stored in a storage system. Typically, a combination of letters, numbers, and separators that uniquely identifies a <i>slot</i> in a storage system.
AGV	Automated Guided Vehicle. A means to transport items using portable robots.
Analysis	See <i>Simulation Analysis</i> .
Animation	A sequence of object movements that are triggered as a simulation runs.
Array	A data structure that consists of an ordered series of elements that are indexed.
Bay (Rack)	A section of a Rack object that represent storage areas along the horizontal axis.
Continuous simulation	A type of simulation where the states of a system change continuously over time, e.g. the filling of a tank with water or other fluid. This is compared to <i>discrete-event simulations</i> where states change at discrete points in time, e.g. a part arriving at a machine to be processed.
Conveyor	A mechanical means to move items between points in space. There are different types of conveyors, all of which can easily be modeled in FlexSim, including: • Belt conveyor, sometimes referred to as a non-accumulating conveyor, behaves as follows. An item travels down the conveyor until the end and then stops if it cannot be removed (downstream object or transport is not

available). When one item stops, the belt stops and then all other items on the conveyor stop in their current location,

- Merge conveyor, combine items from multiple conveyor lines into a single line for further transportation.
- Power and free conveyor, uses a free rail and multiple drive rails to move items loaded on trolleys through a system.
- Roller conveyor, sometimes referred to as an accumulating conveyor, behaves as follows. An item travels along the conveyor until the end and then stops if it cannot be removed (downstream object or transport is not available). Subsequent items on the conveyor continue to flow on the conveyor and stop behind the one in front.
- Slug conveyor, a type of merging where a group of accumulated items are release for further transport as a group, thus items are released in “slugs.”
- Sorting conveyor, transports items to different destinations based on stated criteria and item properties. Sorting typically involve conveyor logic objects such as decision points, stations, and photo eyes.

Dashboard	An area for displaying values of some variable as a model runs, typically the current state of a system or a summary measures (e.g. mean). The display may be in text form, but is typically a graph or chart that updates as a model runs. Dashboards provides a means to view the system dynamics.
Discrete-Event Simulation	The means to consider the dynamics of a system by creating and managing events at discrete points in time. Each event changes one or more states of a system. The resulting state change and modeling logic causes an action or activity to occur.
Domain expert	A person with specialized knowledge or skill in an area of interest.
Dot Notation	A notational convention in object-oriented programming where the elements of an object (variables and methods) are separated by dots, e.g. Processor.Type where Type is a variable attribute of the object Processor.
Downtime	The time period when a resource is not available to perform its basic functions. Downtimes are either planned or unplanned. • Planned downtimes are expected to occur at known times and typically recur on a regular basis. Examples include: shift schedules, operator breaks, and periodic quality checks. Planned downtimes are implemented in <i>FlexSim</i> via the Time Table tool. • Unplanned downtimes occur unexpectedly. Examples include: machine breakdowns and operator absenteeism, Unplanned downtimes are implemented in <i>FlexSim</i> via the MTBF/MTTR tool.
Event	An occurrence at an instance in time. When an event occurs, it causes one or more system states to change and possibly one or more actions.
Experiment	A set of <i>scenarios</i> and <i>performance measures</i> considered for analysis. The simulation model is run for a prescribed number of replications and duration and may involve excluding a <i>warm-up</i> period.

Experimenter	A tool for designing, executing, analyzing, and possibly optimizing simulation models. It provides a means for defining <i>scenarios</i> , <i>performance measures</i> , number of <i>replications</i> , simulated time or duration, and whether a <i>warm-up</i> period is considered. IA direct connection to <i>OptQuest</i> for optimization is provided.
<i>Expert Fit</i>	Software included with <i>FlexSim</i> , developed by Dr. Averill Law, to best fit data to probability distributions. The software is typically used to define input distributions to a simulation model from operational data, but may also be used to characterize output data from a simulation in terms of probability distributions.
Fixed Resource	Objects in a model that are fixed or stationary, e.g. Processor or Combiner. Typically, fixed resources process flowitems.
FlexScript	A scripting language, that is a subset of C++, within <i>FlexSim</i> that can be used to define logic and behaviors via custom coding. FlexScript includes many pre-built functions, or commands, for performing common programming operations.
Flowitem	As the name indicates, an item that flows through a simulation model. Movement is triggered by events and controlled by model logic. Similar to entities or transactions in other simulation software.
Flowitem Bin	A tool in the Toolbox for customizing and creating flowitem classes, including size, shape, graphic, labels, packing method, etc.
Global Table	A systematic means to store information in terms of rows and columns.
Global Variable	User-defined information that can be accessed and updated from anywhere in a FlexSim model at any time during a simulation. Each variable is typed as Integer, Real, String, Array, etc.
Label	A user-defined property, on an object or item, that can be defined, used, and/or updated at any time during a simulation.
Level	Storage spaces along the vertical axis of a <i>Bay</i> in a Rack object.
List	A tool for creating complex flows in a model. Basically, in a certain condition an object pushes information onto a list (referred to as a list entry) and under a different condition an object, using selection criteria, pulls an entry from the list, resulting in an action in a model.
Mobile Resource	see <i>Task Executer</i> .
Model	Representation or abstract of a system and its behavior. A model represents a system's components and the interaction among the components, variability inherent in the system, and the system's dynamic behavior.
Model Fidelity	The degree to which a model represents the real system. While it might appear that more is better; in this case, it is not. A model should only be as detailed as needed to answer the posed question(s). Of course, this is easier said than done; determining the right fidelity comes with practice and experience.
Modeling	See <i>Simulation Modeling</i> .

Modeling Surface	The interface where models are built in 3D space by dragging and dropping objects onto the gridded-surface and locating them in 3D space in terms of their x, y, and z coordinates. In <i>FlexSim</i> , the surface grid's unit of measure is specified as Length Units when a model is started; e.g., if the Length Unit is feet, then each grid unit represents one foot in the x and y directions. Similar modeling environments are available when defining model logic in Process Flow and creating charts and graphs in Dashboards; however, these are just dimensionless work surfaces for organizing work.
Monte Carlo simulation	A means to obtain numerical results based on repeated random sampling.
MTBF/MTTR	Mean Time Between Failures / Mean Time To Repair. In <i>FlexSim</i> , this tool provides the ability to model the reliability of an object in terms of a set of parameters and properties such as: the time between failures (operating time), downtime duration, resources needed, states affecting the time between failures (e.g. clock time versus processing time), etc. Typically, the time between failures and repair times are random variables.
Object	A pre-built, yet customizable, representation of actions commonly found in operations systems, such as planned and unplanned delays (e.g. processing, storage), transportation via fixed (e.g. conveyor) or mobile (e.g. operator) means, combining or separating objects, etc. Its behavior is defined through properties that are specified via its user interface, the <i>Process Flow</i> logic builder, or custom computer code using <i>FlexScript</i> .
Object Library	A list of available objects that can be selected and dragged and dropped onto the modeling or work surface. The primary <i>FlexSim</i> library contains Fixed Resources, Task Executors, Conveyors, Fluids, Modules, etc. User libraries of objects can be developed that contain special user-developed objects. Other libraries are available for the defining logic in Process Flow and creating charts and graphs.
Object Flow Diagram	A diagramming methodology, used during the conceptual design phase of model development, to identify and represent the various system elements and relationships that must be considered by the simulation.
Operational dynamics	The changing behavior and performance of an operations system over time.
Operations system	A collection of elements that transform input into output through a set of related activities and processes that require a variety of resources, such as equipment, material, people, and information. Operations systems are characterized by complex interactions among resources, numerous sources of variability, and exhibit dynamic behavior, i.e. change over time.
<i>OptQuest</i>	A simulation optimization software product, developed and maintained by OptTek Systems, Inc., that provides algorithms and analysis techniques for determining the best input values to obtain the best outcomes.
Path Network	A set of connected nodes that defines a path on which Task Executors (TE) can travel. In <i>FlexSim</i> , the paths can restrict direction of travel, speed and, passing, as well as be straight-line or curved. In contrast to <i>A-Star</i> , which also controls Task

	Executor movement, a Path Network explicitly restricts or limits where a TE can travel; whereas, A* identifies where a TE needs to avoid or cannot travel.
Performance Measure	An output of a simulation model that is of interest for analyzing and understanding the estimated behavior of the system being considered.
Port	A way for <i>flowitems</i> to move in to and out of objects (input ports and output ports). Also, provide a means for objects to communicate (center ports). Whenever two items are connected, a port is created on each object. Theoretically, there is no limit on the number of ports an object may have.
Process Flow	Drag-and-drop, flowchart-like logic builder within <i>FlexSim</i> that is used to specify inter-object or intra-object logic.
Pull logic	A means of flow between objects where the decision as to which object an item is obtained from is determined by the receiving object, not the sending object. See also <i>Push logic</i>.
Push logic	A means of flow between objects where the decision as to which object receives the item is determined by the sending object, not the receiving object. See also <i>Pull logic</i>.
Queueing System	Composed of three main elements - customers, servers, and queues (also referred to as buffers). Customers require service from one or more servers and if a server is not available when the customer demands the service, the customer waits in queue until a server is available. The interaction among the three elements result in the behavior of the system.
Reliability	The probability that an object performs its desired functions under certain conditions for a period of time. Another way to state this is that reliability is the probability that an object does not fail over a period of time. See <i>MTBF/MTTR</i> for the means to specify reliability in <i>FlexSim</i>.
Replication	The repeating of a simulation experiment. Replications are necessary because simulation involve random variables or stochastic parameters.
Routing	The logic that is used to move flowitems between objects. Commonly, items are “pushed” from one object to another, but may also be “pulled” from one object to another. Routing is typically defined on an object’s Flow tab. • Push logic involves deciding which output port will be used and includes the first downstream object that is available or the one with the shortest queue, conditioned on the value of the item’s attribute (label) or a state of the system, etc. • Pull logic options are similar to those used for pushing, but the decision involves which input port on the object should be used to obtain an item.
Run Time	The duration of a simulation run in simulated time, not real time. Time in <i>FlexSim</i> is unit-less; it is given context through the user’s specification of time units (e.g., seconds, minutes, days). If the user specifies the model units as seconds, then a Run Time of 10,000 is 10,000 seconds of simulated time (about 2.8 hours).

Run Speed	How fast a model runs relative to real time. For example, if a model's units are specified as seconds, then a Run Speed of 1000 means the model runs 1000 times faster than real time. Therefore, if a model's Run Time is 10,000 seconds (about 2.8 hours) and Run Speed is set to 1000, then the 2.8 hour simulation will take 10 seconds to run.
Scenario	A combination of model variable settings or characteristics. The values of the variables are changed during experimentation in order to determine their affect on estimated system performance.
Simulation	A process that involves the modeling and analysis of an operations system in order to improve organizational performance.
Simulation Analysis	Simulation models are built for analysis. As such, this is the ways and means of using a simulation model to experiment with and test ideas and alternatives before deciding actions and committing resources. Analyses may be performed within the software or exported to other software. In addition to output analyses, input analyses are needed to define system properties and represent them in a model. Oftentimes, input analyses involve selecting the most appropriate probability distributions to represent aspects of the system, e.g. process times, inter-arrival times, and quality.
Simulation Modeling	The ways and means for representing a system physically (size, distance, speed, etc.) and logically (what, who, when, and where things are done, as well as how much and how long) in order to understand its behavior over space and time and to assess possible consequences of actions, virtually.
Slot (Rack)	A storage space in a Rack object that is a subdivision of a <i>Bay</i> in the horizontal direction.
State	A condition of a system or the value of a system variable, such as whether a resource is busy or idle or how many customers are currently in the system.
Staytime	How long an items stays in an object.
Study Model	A small model that is developed to test and validate a method or concept in isolation before incorporating it into the main model. This good modeling practice is commonly used to develop logic for a single object or set of objects.
Table	A systematic means to store information. In <i>FlexSim</i> , tables can be imported from, or exported to, Microsoft Excel. See either <i>Global Table</i> or <i>Timetable</i> .
Task Executer	A mobile resource that moves about in a model, typically to move flowitems between objects. They include such objects as operator, fork truck, AGV, crane, etc. As the name indicates, the object executes a sequence of tasks that are sent from another object. Tasks include travel between objects, load an item, process an item, etc.
Task Sequence	A set of tasks that a task executer (operator, fork truck, crane, etc.) carries out. Typically, task sequences are requested and sent by fixed resources and involve tasks such as travel between objects, load an item, process an item, etc.

Timetable	A means to define and specify a resource's deterministic availability, such as shift schedule, operator break times, periodic inspections, etc.
Token	The basic component of <i>FlexSim's</i> Process Flow logic. Tokens are analogous to flowitems in 3D in that they flow through activities as a simulation runs. However, tokens are typically more abstract than flowitems since they usually represent logic flow, rather than physical flow. Each token has a set of labels or characteristics.
Toolbox	A type of library that contains modeling tools or aides, such as data tables, time tables, storage systems, etc.
Transfer	Objects that are automatically created in a model whenever a conveyor segment is connected to another conveyor segment or to another type of fixed resource.
Tree	The hierarchical, object-oriented data structure that stores the model data. In <i>FlexSim</i> , all of this data is readily accessible.
Trigger	A place in a model, typically on an object, where optional actions can be defined to occur when certain conditions are met as a model runs, such as: when a flowitem enters/exits an object, a process is finished, a message is received, the model is reset. There are many types of actions that can occur when a trigger is "fired," such as: change an item's appearance (shape, color, etc.), read/write a table or label value, open/close ports, send a message, etc.
Units	Units of measure - time, length, and volume (for fluid-based models only) - that must be specified at the beginning of a simulation.
Validation	The process of determining if a model is accurate enough for its intended use.
Verification	The process of determining if a model has been implemented correctly.
Warm up	A period of time at the beginning of a simulation where statistics are not collected. This allows the simulation to get to a point where the conditions are more representative of those that would occur in the real system. For example, a simulation may start with all resources empty and idle, which may not be typical in the real system, and thus the statistics would be biased low if this early time is included in an analysis.

ABOUT THE AUTHOR

Allen G. Greenwood, Ph.D., P.E.



Currently, Allen is both a Simulation Education Specialist at FlexSim Software Products, Inc. and Professor Emeritus of Industrial and Systems Engineering at Mississippi State University. One of his long-term, foundational professional goals, both in practice and in academe, has been to enhance and increase the application of simulation to support the problem solving and decision making processes.

In addition to his faculty positions at Mississippi State, he was Professor of Engineering Management at Poznan University of Technology in Poland, Professor and Chair of the Department of Engineering Management at Prince Sultan University in the Kingdom of Saudi

Arabia, Professor of Engineering at the American University of Armenia, and Assistant Professor of Management Sciences at Northeastern University and Virginia Tech.

At all of these institutions, he developed and taught courses in systems simulation at the undergraduate, graduate, and professional levels. In addition, he taught courses in operations research / management science, logistics systems design, enterprise systems engineering, project management, statistics, decision analysis, information systems, etc.

Allen's research interests/expertise include the design and analysis of production and project systems; simulation modeling, analysis, and optimization; and the design and application of decision-support systems.

Prior to academe, he held engineering and supervisory positions at American Enka Company and General Dynamics Corporation. During his academic career, he has led or was a principal contributor to numerous projects in industry, mostly in the area of systems simulation. As a result, his professional experience spans a wide variety of domains -- engineering design and development (military aircraft and aerospace), manufacturing and production systems (military aircraft, shipbuilding, automotive, textile fibers, healthcare, electrical systems, material handling systems, consumer products, etc.), and project management. His work has been funded by such organizations as the US Air Force Research Laboratory, Office of Naval Research, Naval Sea Systems Command, NASA, Northrop Grumman Ship Systems, Nissan North America, General Electric Aviation, the Center for Advanced Vehicular Systems (MSU), and Poznan University of Technology.

He has authored or co-authored over 150 creative works, including journal and conference papers, technical reports, software programs, etc. In addition, he is co-author of *Applied Simulation: Modeling and Analysis Using FlexSim*, currently in its fifth edition, and recently authored two primers, one focused on simulation in general and one focused on using *FlexSim* software.

Allen received his B.S.I.E, M.S.I.E, and Ph.D. (Management Science) degrees from North Carolina State University, University of Tennessee, and Virginia Tech, respectively. He is a registered Professional Engineer (retired) in Texas.

Allen may be contacted at allen.greenwood@flexsim.com.